

Competitive Programming

Section 9: Greedy I - Local Choices And Exchange

Competitive Programming Course

2026-07-17

Table of contents

1	Why greedy is dangerous and useful	3
2	The shape of this section	4
3	How to recognize a greedy candidate	5
4	Vocabulary	6
5	Worked example 1: intervals	7
6	Worked example 2: boats	8
6.1	Problem model	8
6.2	Exchange proof for the pairing case	9
7	Worked example 3: deadlines	10
7.1	Why task C forces a removal	10
7.2	Why discard the longest task	11
8	Heap primer for deadline scheduling	12
9	A proof checklist for greedy	14
10	Common wrong greedy rules	15
10.1	Activity selection	15
10.2	Rescue boats	15
10.3	Deadline schedule	16
11	Implementation notes in C++	17
12	Implementation notes in Python	18
13	Local judge workflow	19
14	Submission checklist	20
15	Exercises	21
15.1	A. Activity Selection	21
15.2	B. Rescue Boats	21
15.3	C. Deadline Schedule	21

1 Why greedy is dangerous and useful

A greedy algorithm makes a decision now and does not revisit it.

That is exactly why greedy is attractive in contests:

- the code is often short;
- the running time is usually sorting plus a scan;
- the idea may be visible from the statement.

It is also why greedy is dangerous. Many wrong greedy solutions pass samples because the first few examples are too small to expose the bad local choice.

The central question is not:

`Does this local choice feel natural?`

The central question is:

`Can every optimal solution be transformed to one that makes this local choice?`

That transformation is called an **exchange argument**.

2 The shape of this section

This section studies three common greedy choices.

Pattern	Local choice	Typical data structure	Local exercise
Earliest finish	Keep the interval that ends first	Sort by end time	Activity Selection
Pair with largest remaining	Decide the fate of the heaviest item	Sort and two pointers	Rescue Boats
Drop largest cost	Keep many items, remove worst offender	Sort and max-heap	Deadline Schedule

These are not isolated tricks. They are templates for proof.

3 How to recognize a greedy candidate

Greedy often appears when the statement contains:

- choosing a maximum-size subset;
- minimizing the number of groups, machines, boats, rooms, or days;
- deadlines or intervals;
- a sorted order that seems to reveal local conflicts;
- no need to output the exact set, only the optimum value.

The constraints also matter. If n is up to 200000, exhaustive subset search is impossible. If the structure has only one dimension, such as time, weight, or deadline, sorting is usually worth trying.

The usual thought process:

1. What local conflict prevents taking everything?
2. If I must resolve one conflict, which item is safest to keep or discard?
3. Can I prove that an optimal answer exists after making that choice?
4. After the choice, does the remaining problem have the same form?

4 Vocabulary

Greedy choice. A decision made using current information, without backtracking.

Exchange argument. A proof that takes an optimal solution that may not use our greedy choice and swaps one or more elements so that it does use the greedy choice, without making the solution worse.

Canonical optimal solution. An optimal solution with an extra nice property, such as “among all optimal interval schedules, choose one whose first interval ends earliest.” Greedy proofs often show that this canonical solution agrees with the greedy algorithm.

Invariant. A fact that remains true after every step. Greedy scans often maintain an invariant like “the chosen set is as good as any set of the same size among processed items.”

Counterexample search. The habit of trying small inputs that make the local choice fail. Before trusting a greedy rule, attack it.

5 Worked example 1: intervals

Suppose we want the maximum number of non-overlapping intervals:

```
A: [1, 10)
B: [1, 3)
C: [3, 5)
D: [5, 7)
```

The longest interval starts earliest, but choosing it gives only one activity.

```
time: 1 2 3 4 5 6 7 8 9 10
A:  [-----)
B:  [----)
C:      [----)
D:          [----)
```

The correct greedy rule is:

Choose the interval with the earliest finishing time among compatible intervals.

Why? Finishing earlier leaves at least as much room for future intervals as any other first choice.

Trace:

```
sort by end:
B [1,3), C [3,5), D [5,7), A [1,10)

last_end = -infinity
take B, last_end = 3
take C, last_end = 5
take D, last_end = 7
skip A

answer = 3
```

The proof is not “it seems to leave room.” The proof is an exchange:

1. Let g be the interval with earliest end.
2. Take an optimal solution and look at its first interval x .
3. Since g ends no later than x , replacing x by g cannot create an overlap with the rest of that optimal solution.
4. Therefore some optimal solution begins with g .
5. After choosing g , solve the remaining compatible intervals the same way.

6 Worked example 2: boats

6.1 Problem model

There are n people. Person i has weight $w[i]$. Every person must take exactly one boat; a boat may carry one or two people, and the sum of the weights in that boat must not exceed the weight limit L . No person may be split between boats. Assume every individual weight is at most L , so a solution always exists.

Goal: use the minimum possible number of boats.

The input order has no meaning: people do not arrive in that order and boats do not have labels. We may therefore sort the weights. Do not confuse this with maximizing the number of pairs. Pairing two people is useful only when it does not prevent a later person from fitting into a boat; the quantity to minimize is the total number of boats.

For example, with $L = 5$ and weights 1 2 2 3, the answer is 2: one boat carries (1, 3) and the other carries (2, 2).

Weights:

```
1 2 2 3
L = 5
```

Sorted:

```
index:    0  1  2  3
weight:   1  2  2  3
          ^      ^
          left   right
```

left is the lightest person not assigned to a boat yet. **right** is the heaviest unassigned person. The unassigned people are always exactly the sorted slice from **left** through **right**.

The heaviest person must be placed on some boat. There are only two cases:

- If the lightest person can ride with the heaviest, pair them.
- Otherwise, nobody can ride with the heaviest, so the heaviest rides alone.

Trace:

```
1 + 3 <= 5, pair them. boats = 1
2 + 2 <= 5, pair them. boats = 2
```

6.2 Exchange proof for the pairing case

Let h be the heaviest remaining person and l the lightest. Assume that $h + l \leq L$. We show that some optimal solution puts h and l together.

- If an optimal solution already has boat (h, l) , there is nothing to prove.
- If h is alone, replace h 's boat and l 's boat by (h, l) and, when l had a partner q , a one-person boat (q) . This uses no more boats.
- Otherwise h shares a boat with a person p . If l is alone, replace (h, p) and (l) by (h, l) and (p) . If l shares a boat with q , replace the two original boats

(h, p) and (l, q)

by

(h, l) and (p, q) .

The first new boat is feasible by the assumption $h + l \leq L$. For the second, (h, p) was feasible, so $h + p \leq L$. Since h is the heaviest remaining person, $q \leq h$; therefore $p + q \leq p + h \leq L$. The second new boat is feasible as well.

Every case preserves or improves the boat count, so an optimal solution exists that pairs h with l . This justifies moving both pointers when they fit.

7 Worked example 3: deadlines

Each task has a duration and a deadline. We want to finish as many tasks as possible.

task	duration	deadline
A	1	3
B	4	5
C	2	6
D	3	7
E	1	7

Sort by deadline:

A (1,3), B (4,5), C (2,6), D (3,7), E (1,7)

Process tasks in that order, tentatively accepting each task:

```
take A: total = 1 <= 3
take B: total = 5 <= 5
take C: total = 7 > 6    remove B (duration 4), total = 3
take D: total = 6 <= 7
take E: total = 7 <= 7

kept tasks: A, C, D, E
answer = 4
```

7.1 Why task C forces a removal

This is not because C happens to be written fourth. We sorted by deadline, so every task currently kept at that moment has deadline at most 6:

```
A: deadline 3
B: deadline 5
C: deadline 6
```

Their total work is $1 + 4 + 2 = 7$. By time 6, a single worker can complete only six minutes of work. Therefore **no ordering** of these three tasks can finish all three by their deadlines. For example, deadline order gives

```
A: 0..1    finishes by 3
B: 1..5    finishes by 5
C: 5..7    misses deadline 6
```

We must discard at least one of these three tasks. Before adding C, tasks A and B were feasible, so discarding C itself would restore a feasible two-task set. Thus discarding exactly one task is sufficient, and we should choose the discard that leaves the best chance for later tasks.

7.2 Why discard the longest task

Removing one task keeps the number of accepted tasks unchanged at two. Among all one-task removals, remove the largest duration: it leaves the smallest total workload.

Remove the longest duration.

```
accepted durations: 1, 4, 2
remove 4
remaining total = 3
kept tasks: A, C
```

The other removals leave totals 6 and 5, which are also feasible now. They are worse because they keep more work than necessary. Removing B leaves only 3 minutes of work, so task D (three more minutes) and then task E (one more minute) can both still be accepted. This is the payoff of the rule: we remove an **older** task to make room for later tasks, rather than merely undoing the newest addition.

More precisely, before the new task we had $\text{old_total} \leq \text{deadline}$. After adding a duration d , the heap maximum M is at least d , because the new task is inside the heap. Therefore

```
new_total - M ≤ new_total - d = old_total ≤ deadline.
```

So one removal always repairs the current deadline violation. It also leaves the minimum possible total among all equally large accepted sets, making every future deadline easiest to meet. If the removed task was old, every earlier prefix only becomes shorter; if it was new, every earlier prefix is unchanged. The already-checked earlier deadlines remain valid.

8 Heap primer for deadline scheduling

The deadline problem needs a data structure that can repeatedly answer:

What is the longest duration among the tasks I have accepted so far?

A **heap** is a container that can quickly return one extreme element:

- a max-heap returns the largest element;
- a min-heap returns the smallest element.

For this section, we use a max-heap of chosen task durations.

The operations we need are:

```
push(x):      insert duration x
top():        read the largest duration
pop():        remove the largest duration
size():       number of chosen tasks
```

Each push and pop costs $O(\log n)$. Reading top costs $O(1)$.

Example:

```
chosen durations: 1, 3, 2
max-heap top:     3

insert 4
chosen durations: 1, 3, 2, 4
max-heap top:     4

pop top
chosen durations: 1, 3, 2
```

This matches the greedy rule:

If total time exceeds the current deadline, remove the longest chosen task.

In C++, `priority_queue<long long>` is a max-heap by default:

```
priority_queue<long long> chosen;
chosen.push(duration);
longest = chosen.top();
chosen.pop();
```

In Python, `heapq` is a min-heap. To simulate a max-heap, store negative durations:

```
heapq.heappush(chosen, -duration)
longest = -chosen[0]
removed = -heapq.heappop(chosen)
```

In the reference solution, we write:

```
total += heapq.heappop(chosen)
```

because the popped value is negative. Adding it subtracts the removed duration.

9 A proof checklist for greedy

Before submitting a greedy solution, fill this out in your head.

Greedy choice:

What exact item do I keep, pair, or discard?

Local conflict:

What constraint forced me to decide?

Exchange:

If an optimal solution does not make my choice, how do I modify it?

No loss:

Why is the modified solution still valid and at least as good?

Recursion or invariant:

After this step, why is the remaining scan still the same kind of problem?

If you cannot answer the exchange question, your greedy idea is not ready.

10 Common wrong greedy rules

10.1 Activity selection

Wrong rule:

```
Pick the shortest interval.
```

Counterexample:

```
[2,3) is shortest  
[1,2), [2,4), [4,5) gives three intervals
```

Shortest interval alone does not say anything about how much useful space is left on both sides.

Wrong rule:

```
Pick the interval that starts earliest.
```

Counterexample:

```
[1,100)  
[2,3), [3,4), [4,5)
```

The earliest starter can block many later intervals.

10.2 Rescue boats

Wrong rule:

```
Pair adjacent sorted weights from lightest to heaviest.
```

Counterexample with $L = 10$:

```
1 2 8 9  
adjacent pairs: (1,2), 8, 9 -> 3 boats  
greedy pairs:   (1,9), (2,8) -> 2 boats
```

The heaviest people are the bottleneck. Decide them first.

10.3 Deadline schedule

Wrong rule:

Always take the shortest available task first.

Counterexample:

duration	deadline
2	2
2	2
100	100

Taking short tasks first is not enough. Deadlines determine the order in which constraints become active.

11 Implementation notes in C++

Use sorting aggressively:

```
sort(v.begin(), v.end(), [](const auto &a, const auto &b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first < b.first;
});
```

For deadline scheduling, use `priority_queue<long long>` as a max-heap of chosen durations:

```
priority_queue<long long> chosen;
long long total = 0;

for (auto [duration, deadline] : tasks) {
    total += duration;
    chosen.push(duration);
    if (total > deadline) {
        total -= chosen.top();
        chosen.pop();
    }
}
```

Use `long long` for total time and deadlines. Even when each value fits in `int`, the sum over many tasks may not.

12 Implementation notes in Python

Python sorting is stable and expressive:

```
intervals.sort(key=lambda p: (p[1], p[0]))
```

Python's `heapq` is a min-heap. To simulate a max-heap, push negative values:

```
heapq.heappush(chosen, -duration)
if total > deadline:
    total += heapq.heappop(chosen)
```

The popped value is negative, so adding it decreases `total`.

For large input, read all tokens:

```
data = list(map(int, sys.stdin.buffer.read().split()))
```

13 Local judge workflow

Run all Section 9 exercises:

```
python3 sections/09_greedy_exchange/check.py
```

Run the reference solutions:

```
CP_TARGET=solution python3 sections/09_greedy_exchange/check.py
```

Each problem has fixed tests and randomized tests. The randomized tests compare the reference greedy solution against a brute-force solver on small instances. That is especially useful for greedy topics, where a wrong rule often survives many hand-written samples.

14 Submission checklist

Before submitting a greedy solution:

- State the greedy rule in one exact sentence.
- Check whether sorting key and tie-breakers match the proof.
- Try at least two counterexamples against alternate greedy rules.
- Confirm endpoint conventions for intervals, such as $[s, e)$.
- Use `long long` or Python integers for accumulated sums.
- Stress small cases against brute force when possible.
- Keep the proof and implementation aligned: do not prove one rule and code another.

15 Exercises

15.1 A. Activity Selection

Practice earliest-finish greedy and a clean exchange proof.

15.2 B. Rescue Boats

Practice deciding the bottleneck item first and using two pointers.

15.3 C. Deadline Schedule

Practice a greedy scan with a heap and the “remove the most expensive accepted item” exchange.